UiT The Arctic University of Norway

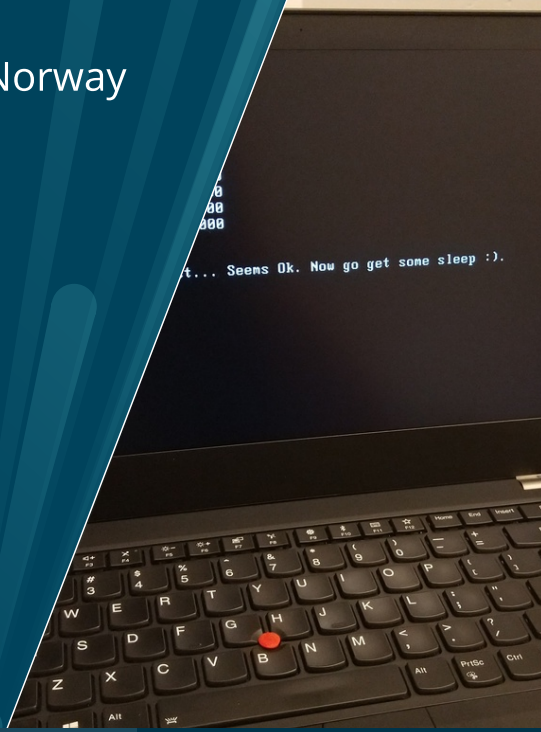
Project 3

Threading and Synchronization

Mike Murphy

UiT INF-2203

Spring 2026



P3 Threading and Synchronization

- P1: Boot and run a program
- P2: Protect against misbehaving processes
- **P3: Multithreading**

1. Implement threads
   - Save and restore CPU state
   - Allow processes to be multi-threaded
2. Enable preemption
   - Switch threads via timer interrupt
3. Implement mutexes
   - Protect critical sections with mutexes
4. Dine with the philosophers
   - Contemplate the Dining Philosophers problem

What's New?

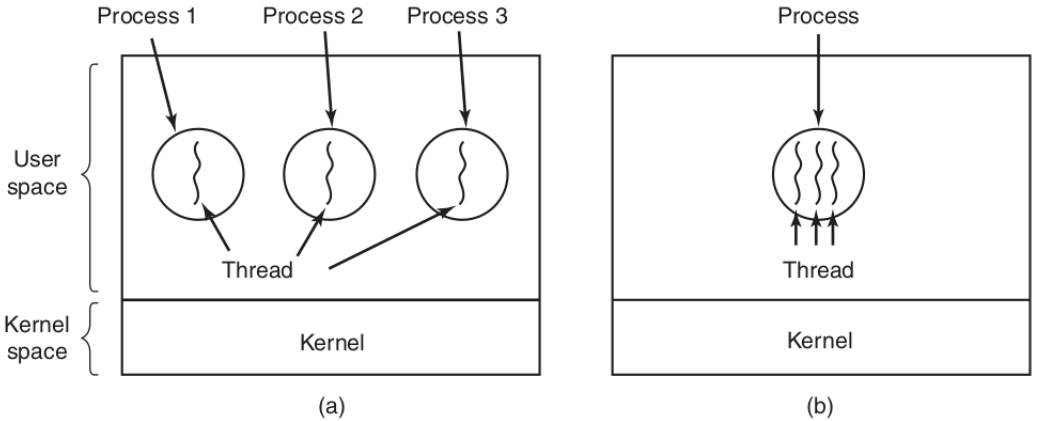
New Features

- Threading APIs
  - User-side APIs for threads
  - Skeleton for mutex API
  - Skeleton for scheduler in kernel
- Hardware interrupts
  - Driver for interrupt controller (PIC)
  - Driver for interval timer (PIT)
- Bug fixes
  - Fix "every-other-run" bug
- Other improvements
  - Less verbose logging
  - New extra-verbose level: TRACE

New Test Processes

- `threads`
  - Spawns and joins threads
- `datarace`
  - Creates a data race, tests mutexes
  - Similar to "Too Much Milk"
- `philosophers`
  - The famous Dining Philosophers
  - Demonstrates deadlocks

Processes vs Threads



(a) Three processes each with one thread. (b) One process with three threads.[1]

[1] From Andrew Tannenbaum and Herbert Bos, *Modern Operating Systems* 4th ed.

Thread API

- New in mulibc: Simplified C11 <threads.h> API
  - We provide the user-side
  - You implement thread support and syscalls in Munix
- `thrd_create`: create a new thread in process
  - Linux syscall: `clone`
  - Munix syscall: `thrd_create`
- `thrd_exit`: exit a thread
  - Linux syscall: `exit` (exit process with `exit_group`)
  - Munix syscall: `thrd_exit` (exit processes with `exit`)
- `thrd_join`: wait for a thread to exit
  - Linux syscall: (wait on *futex*)
  - Munix syscall: `thrd_join`
- Test program: threads.c

Special Thread-Create Syscall Functions

- Thread creation needs a wrapper function
  - Set stack pointer for new thread
  - Call requested thread function
  - Exit thread when thread function returns (like `_start` does for `main`)
- Linux: `syscall_clone_linux`
  - Linux `clone` syscall is similar to `fork`
  - Both threads return to the same point
  - Return value determines parent vs child thread
- Munix: `syscall_thrd_create_munix`
  - Munix `thrd_create` syscall is simpler
  - New thread starts at address we give it
  - Which for us is our `_start`-like wrapper
- You don't have to write these functions, but you should understand them

Implementing Thread Switching

- Some things can be done in C
  - Set kernel stack pointer in TSS (`cpu_user_kstack_set`)
  - Check if thread to switch to is new or resume
  - If new, can reuse `cpu_user_start` from P2
- Some things must be done in assembly language
  - Anything that directly manipulates the stack pointer
  - We want to write core save/restore functions we can call from C

Core Thread Switch

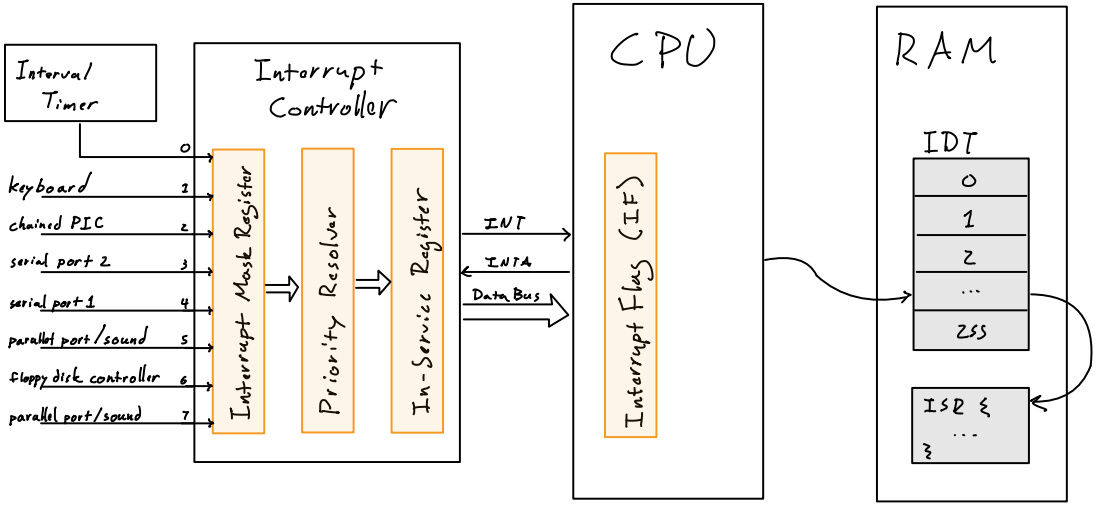
Suggested Save/Restore API

- Inspired by C Standard Library <setjmp.h>
  - Struct `jmp_buf` holds state
  - `setjmp` saves state to `jmp_buf`
  - `longjmp` jumps back to state in `jmp_buf`
- Struct `cpu_task_save` holds state
- Function `cpu_task_save`
  - Save important state in task save struct
  - Return 0
- Function `cpu_task_restore`
  - Restore important state from task save struct
  - Return to caller of `cpu_task_save` with value 1

Hints

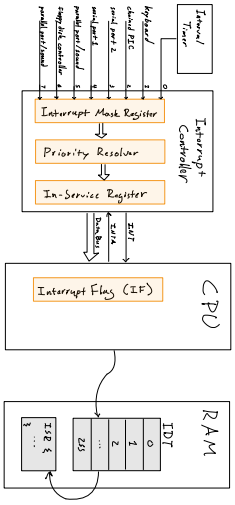
- Need to understand function call ABI
  - Where are parameters?
  - Where is return address?
  - What registers need to be saved?
- Look at other ASM fns
  - Thread create wrappers
  - `_start` for processes
  - `_start` for kernel

PC Hardware Interrupt Support



Interrupt Requests: Devices → PIC → CPU → IDT

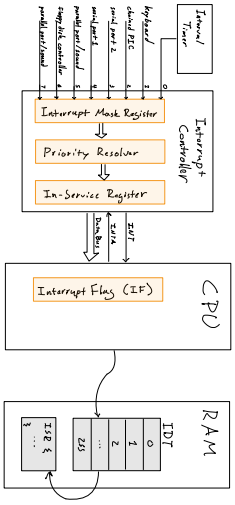
PC Hardware Interrupt Support: Explanation



- Devices make *Interrupt Requests (IRQs)*
  - 0 = timer
  - 1 = keyboard
  - …

Devices → PIC → CPU → IDT

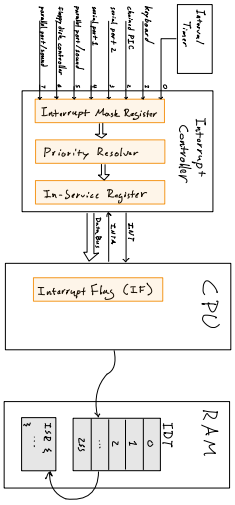
PC Hardware Interrupt Support: Explanation



Devices → PIC → CPU → IDT

- Devices make *Interrupt Requests (IRQs)*
  - 0 = timer
  - 1 = keyboard
  - …
- to *Programmable Interrupt Controller (PIC)*
  - Chooses highest priority IRQ
  - Marks IRQ as *In Service*
    - Ignores repeats of this IRQ during service
  - Sends IRQ to CPU
    - Maps IRQ (0–7) to *interrupt vector* (0–255)
    - Typically IRQ 0 maps to interrupt 32

PC Hardware Interrupt Support: Explanation

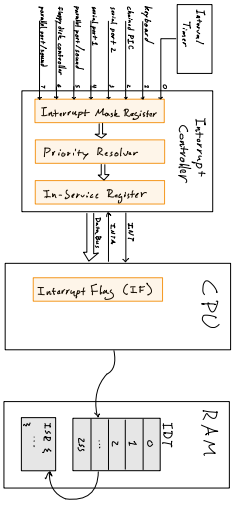


Devices → PIC → CPU → IDT

- Devices make *Interrupt Requests (IRQs)*
  - 0 = timer
  - 1 = keyboard
  - …
- to *Programmable Interrupt Controller (PIC)*
  - Chooses highest priority IRQ
  - Marks IRQ as *In Service*
    - Ignores repeats of this IRQ during service
  - Sends IRQ to CPU
    - Maps IRQ (0–7) to *interrupt vector* (0–255)
    - Typically IRQ 0 maps to interrupt 32
- to CPU
  - Gets signal and vector number
  - Finds routine in *Interrupt Descriptor Table (IDT)*
  - Calls *Interrupt Service Routine (ISR)*

Three Chances to Disable an Interrupt

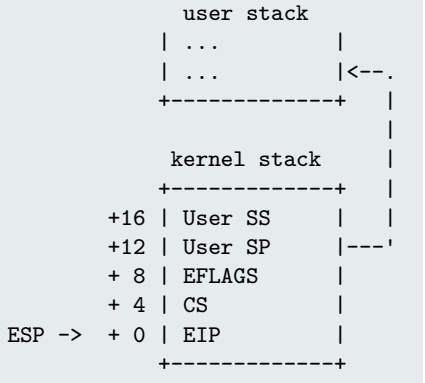
1. PIC: Interrupt Mask Register
   - Ignore specific devices
2. PIC: In-Service Register
   - Marks IRQ as *in service* on send to CPU
   - PIC ignores that IRQ while in service
   - CPU seds *End of Interrupt (EOI)* when finished
3. CPU: Interrupt Flag (IF)
   - In EFLAGS register
   - Clear to ignore all external interrupts
   - CPU clears flag at interrupt start
   - Flag restored when EFLAGS is restored on `iret`



Devices → PIC → CPU → IDT

Focus: Interrupt Flag Cleared During Handler

Interrupt Stack Frame



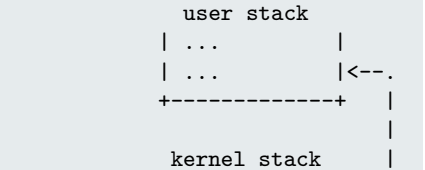
Interrupt Flag

EFLAGS bit 9 is *Interrupt Flag (IF)*

- On interrupt, EFLAGS is pushed to stack
  - CPU then clears Interrupt Flag
  - So, interrupts are off during ISR
- When ISR returns (IRET instruction)
  - Previous EFLAGS is restored
  - So, interrupts are turned back on again

Focus: Interrupt Flag Cleared During Handler

Interrupt Stack Frame



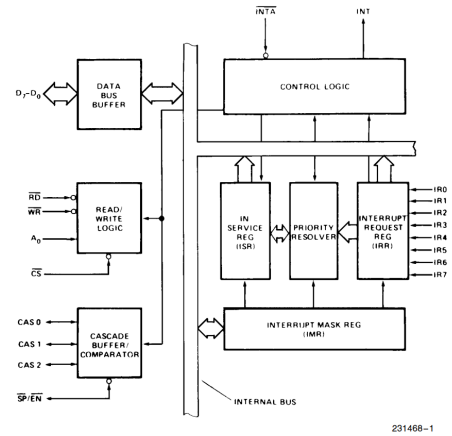
Interrupt Flag

EFLAGS bit 9 is *Interrupt Flag (IF)*

- On interrupt, EFLAGS is pushed to stack
  - CPU then clears Interrupt Flag
  - So, interrupts are off during ISR
- When ISR returns (IRET instruction)
  - Previous EFLAGS is restored
  - So, interrupts are turned back on again
- Possible to turn on interrupts during ISR
  - *Reentrant*
  - Our kernel is NOT reentrant
  - In-kernel code will not be interrupted

Focus: In-Service Register and End of Interrupt (EOI)

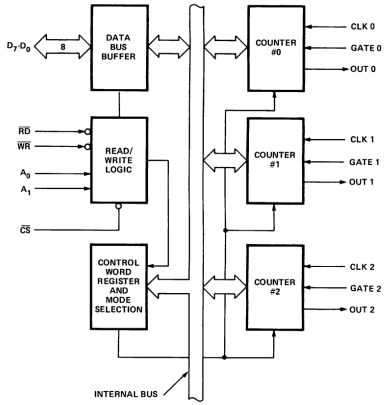
- Interrupt controller marks current IRQ as *in service*
- If another IRQ comes in from same device, it will be ignored
  - Problem if interrupt is not handled fast enough
  - e.g. Lose a key press
- CPU must signal back to interrupt controller when it is finished
  - *EOI: End of Interrupt* signal
  - Clears “in service” bit
  - Allows more IRQs through
- If you never send the EOI, you will not see that IRQ again
  - “Why is my timer only firing once??”



Intel 8259 PIC block diagram[2]

From Intel 8259A Datasheet

Timer Interrupt for Preemption



Intel 8253 PIT block diagram[3]

[3] From Intel 8080 User's Manual

- *Programmable Interval Timer (PIT)* chip
  - Internal oscillator w/ base freq of 1.19 MHz
  - Related to American TVs (NTSC)
  - Can drive signals by dividing that frequency (fire every *n* ticks)
- Three signal channels with own divisors:
  - **Channel 0 → IRQ 0 = Timer interrupt**
  - Channel 1 → DRAM refresh (in early PCs)
  - Channel 2 → PC speaker (before sound cards)
- Timer interrupt
  - Preempt current thread
  - Switch to another thread
  - This is how we get *time slices*
  - We will aim for 1000 time slices per second

Let's Try Something IRL

Let's have 2 people act like threads.
Each of you:

1. Look at shared count on whiteboard
2. Turn around to your side of whiteboard
3. Write down the number you saw + 1
4. Turn back to shared area
5. Erase current count and write your count
6. Repeat 10 times

Let's see what happens...

datarace.c: A Tale of Three Updates

Read, Work, Update

```
unsigned shared_split = 0;

unsigned tmp = shared_split;
if (tmp % 2 == 0)
    asm volatile("");
shared_split = tmp + 1;
```

```
        mov     0x50402c,%eax
        test    $0x1,%al
    /-- jne     1f
1: \-> inc     %eax
        mov     %eax,0x50402c
```

One-Line Increment

```
unsigned shared_oneline = 0;

shared_oneline += 1;
```

```
    incl    0x504028
```

Atomic Increment

```
Atomic unsigned
    shared_atomic = 0;
shared_atomic += 1;
```

```
    lock incl 0x504024
```

Results: Linux

```
Expected result:    2000000
shared_split:       1473021
shared_oneline:     1509222
shared_atomic:      2000000
```

Results: Munix

```
Expected result:    2000000
shared_split:       1978763
shared_oneline:     2000000
shared_atomic:      2000000
```

Basic Primitive: Atomic Flag, Test-and-Set

Primitive Atomic Op

Atomic test and set

- Provided by modern CPUs
- Set flag to 1, return previous value
- If previous value was 1
  - Flag was already set, by someone else
  - Flag is still set
  - You must wait
- If previous value was 0
  - Flag was just now set, by you!
  - Your turn!

Code

```
#include <stdatomic.h>

atomic_flag flag;

int already_set =
    atomic_flag_test_and_set(&flag);
if (!already_set) {
    /* ... */
}
```

```
    mov     $0x1,%al
    xchg    %al,(%edx)
    test    %al,%al
    je      /* ... */
```

Implementing Mutexes

In Precode

- src/lib/mulibc/threads.h
  - Simplified implementation of C11 threads API
  - Mutex implementation is a no-op
  - You must complete it
- datarace.c
  - mutex-protected versions of split update and one-line update
- philosophers.c
  - runs on mutexes
- Can test both on Linux and Munix

You're going to need

- <stdatomic.h>
  Provided by compiler
  - Atomic flag
  - Test-and-set
- Yield syscall (?)

philosophers.c: The Dining Philosophers

In precode:

```
philosophers - the dining philosophers
    -n N    set number of philosophers (default 3)
    -c C    set loop count (default 100)
    -s S    set slowdown (busy wait 2^S loops, default 24)
    -l      make one philosopher left-handed
    -e      exit when the first philosopher finishes
    -h      display help

example
   philosophers -n 5
```

- Experiment with this
- Revisit Loic's lecture
- Read the relevant sections of textbook

Project 2 Tasks Summary

We Provide

- User-side APIs for threading, skeleton for mutexes
- Drivers for interrupt controller (PIC) and timer (PIT)
- Additional skeleton code in kernel

Your Task: **Implement Threading, Preemption, and Mutexes**

1. Implement threads
2. Enable preemption and scheduling
3. Implement mutexes
4. In report: describe details of Data Race and Philosophers programs

1. Implement threads

- **Test: `threads` process should run smoothly**
- Complete `thread` struct
  - What fields does a thread need?
  - What is the difference between a *thread* vs a *process?*
- Change process startup again
  - start process with a single main thread
- Implement low-level CPU save/restore
  - `struct cpu_task_save`, `cpu_task_save`, `cpu_task_restore`
  - This is the mind-bending part
  - Need to really understand thread state: registers, stack, and stack frames
- Implement threading syscalls
  - Thread create, exit, and join

2. Enable preemption and scheduling

- **Test: `datarace` program should demonstrate data races**
  - If your threads all run sequentially
    the `datarace` program will not lose any updates
  - With timer-based preemption enabled
    - Threads can be interrupted at any time
    - Unprotected access to shared variables leads to race conditions
- Wire up timer interrupt
  - How do timer interrupts get to the CPU?
  - When are interrupts enabled/disabled in our code?
  - When should you send the EOI (End of Interrupt) signal in the timer ISR?
- Finish scheduler implementation
  - On timer interrupt: preempt thread and switch to next thread
  - Scheduler: simple round robin is fine
  - What other scheduling algorithms are out there?

3. Implement mutexes

- **Test: `datarace` and `philosophers` should run more smoothly**
  - `datarace`: protected values should stop showing lost updates
  - `philosphers`: output should not get scrambled
- Need to implement user-side API
  - Can test your implementation on both Linux and Munix
  - What are the differences in how Linux and Munix (+QEMU) execute code?
- Implementation: Ok to implement as simple spinlock
  - Should you just spin until preempt, or is it better to yield?
  - What would it take to implement blocking/unblocking?
- Hint: `<stdatomic.h>` provides `atomic_flag`
  - Why are atomic operations important?
  - What is a compare-and-swap? Why is it important?

4. Details of Data Race and Philosophers programs

In your report: discuss nuances in `datarace` and `philosophers`

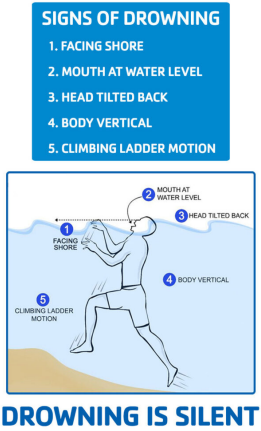
- `datarace`
  - Why do updates sometimes get lost?
  - What are the differences between the five updates in this program?
  - What x86 instructions do the different updates compile to?
    - Especially "oneline" vs atomic?
    - Why is this important?
  - Why does the "one-line" update only lose updates on Linux, not Munix?
- `philosophers`
  - Why does this program deadlock by default?
  - What does the `-l` (left-handed) option do?
  - Why does it prevent deadlocks?
  - **What are the four conditions necessary for deadlock?**
  - With `-l`, why does one of the philosophers eat more than the others?
  - How could you make the food distribution more fair?

If It's Too Easy: Extra Challenges

- Implement more sophisticated mutexes
  - Implement a blocking mutex with its own block queue
  - Expand scheduler to handle this case
  - Implement a *recursive* mutex
  - Look into Linux *futexes*
- Write more test programs
  - Update philosophers implementation to be more fair
  - Implement and test semaphores?
  - We have interrupts now. Write a keyboard driver? A game?
- "Yeah I read theory"
  - Read Dijkstra's original paper on semaphores:
    "Cooperating Sequential Processes" (1965).
    EWD-123 in UTexas archive, https://www.cs.utexas.edu/~EWD/index01xx.html

If It's Too Hard: Ask For Help

- Ask your TAs
- Come to the colloquiums
- Ask your fellow students
- Ask on Discord
- DO NOT ask an LLM



Drowning doesn't look like drowning[4]

[4] Safety poster from Kenosha YMCA

Two Ways to Continue

1. **Continue** building on your solution
   - Git branch `main`
   - Starts with P1 precode, adds only new precode (no solutions!)
   - Should *(should!)* merge cleanly with your projects: `git merge main`
   - More challenging, but more rewarding. OS is more *yours*
   - More chances for bugs. Will have to do more troubleshooting
2. **Start fresh** with common project base
   - Git branches: `p2-pre`, `p3-pre`, `p4-pre`
   - Starts with my solution to last project, then adds new precode
   - Check out, do not merge: `git checkout p2-pre` (or `p3-pre`, `p4-pre`, etc.)
   - Use common project base, focus only on current project
   - A more straightforward project experience

- Remember: This is new code. **There will be bugs either way!**

Hand-In Format: Same as Last Time

- Design reviews mid-way
  - Meet with TA
  - Convince them you have started and have a plan for solving this
- Hand in: Code and Report
  - Zip up code tree + git repo, upload to Canvas
    - Be sure to clean your code tree: `make clean`
    - DO NOT include cross compiler or bootloader
    - DO NOT include `out/` directory
    - DO include `.git/` directory
    - Zip should be a handful of Megabytes
    - Cross compiler will balloon it to Gigabytes (no thanks!)
  - Report
    - See Report Guidelines doc for detail
    - Upload report PDF to Canvas separately from code Zip

Extra Material

- Project 3 Files List
- More Info and Links

P3: New in precode

new	pre	task	file	desc
!	234	13	lib/mulibc/threads.[ch]	User-side thread API
!	45	34	lib/arch/i386/cpu_task_save.S	Low-level task switch
!	253		lib/drivers/chip/intctl_8259.[ch]	PIC driver
!	163		lib/drivers/chip/pit_8253.[ch]	PIT driver
!	94		kernel/scheduler.[ch]	Scheduler
!	80		process/threads.c	Basic thread test
!	138		process/datarace.c	Race condition demo
!	255		process/philosophers.c	Dining philosophers

P3: Updated in precode

new	pre	task	file	desc
	29	4	lib/arch/i386/cpu.h	Read CPU timestamp
	6		lib/core/errno.h	Thread error codes
	210		lib/sys/	Thread-create wrappers
	87	8	kernel/interrupt.c	IRQ init, handling
	13	3	kernel/kernel.[ch]	Stack bug fix, init
	159	126	kernel/process.[ch]	Threading precode

P3: Task files

new	pre	task	file	desc
	1	3	kernel/syscall_dispatch.c	Wire up thread syscalls
	159	126	kernel/process.[ch]	Implement syscalls
	29	4	lib/arch/i386/cpu.h	CPU state save struct
!	45	34	lib/arch/i386/cpu_task_save.S	Low-level task switch
	5	2	lib/arch/i386/cpu_interrupt.[ch]	Declare IRQ entry fn
	1	1	lib/arch/i386/x86_seg.[ch]	Enable Interrupt Flag
	13	3	kernel/kernel.[ch]	Init PIC and PIT
	87	8	kernel/interrupt.c	Handle timer IRQ
!	234	13	lib/mulibc/threads.[ch]	Implement mutexes

Estimated project workload: 194 lines

Extra Material

- Project 3 Files List
- More Info and Links

Useful Resources I

- Textbook: *Modern Operating Systems* 4th ed.
  - Section 2.5.1: The Dining Philosophers Problem
  - Section 6.2.1: Conditions for Resource Deadlocks
  - Section 6.5: Deadlock Avoidance
- x86 interrupt handling:
  - Intel® 64 and IA-32 Architectures Software Developer Manuals,
    especially Volume 3, System Programming Guide, Chapter 6, Interrupt and Exception Handling
  - AMD64 Architecture Programmer's Manuals,
    especially Volume 2, System Programming, Chapter 8, Exceptions and Interrupts
- PC interrupt and timer chips:
  - OSDev Wiki: 8259 PIC:
    Overview of the Programmable Interrupt Controller, which communicates external hardware Interrupt Requests (IRQs) to the CPU

Useful Resources II

- OSDev Wiki: Programmable Interval Timer:
  Overview of the timer that raises the IRQ we use for preemption

- Assembly language:
  - A Guide to Programming Pentium/Pentium Pro Processors (PDF in repo)
    A good introduction to get you started (or refreshed) on assembler. Covers both Intel and AT&T syntax.
  - x86 Assembly Guide:
    Basics of 32-bit x86 assembly language programming.
  - System V ABI, i386 Supplement (PDF in repo)
    Documentation of the C function calling conventions and more.

- Threads:
  - CPPReference: C11 <thread.h> API:
    Reference for C11 threading and mutex APIs, which we mimic in our code

Useful Resources III

- Programming with Threads (PDF in repo)
  A tutorial on threads in general